PRODUCT REQUIREMENTS DOCUMENT

Kodewar digital learn

Detailed product specification for an AI application builder and engineering team

Document property	Value
Document version	1.2 — consolidated discovery draft
Product type	Single-company digital learning and candidate management platform
Primary users	One Super Admin and enrolled Candidates
Portals	Admin Portal and Candidate Portal
Status	Detailed draft for review and implementation planning
Prepared	03 November 2026
Intended audience	AI application builder, product owner, designers, developers, QA

Purpose. This document describes the product scope, user journeys, functional requirements, business rules, data expectations, technical implementation constraints, responsive behavior, and acceptance criteria. It is intended to be used as the authoritative build brief for implementing the first production-ready release.

Important implementation directive. Build and verify one functional module at a time. Do not move to the next module until the current module works end-to-end, passes its acceptance checks, and does not cause regressions or page crashes.

Document control and usage

Item	Details
Product name	Kodewar digital learn
Version	1.2
Scope	Single-company platform; no tenant registration or tenant switching in MVP
Portals	Admin and Candidate
Administrative authority	Exactly one Super Admin account in the initial release
Out of scope	Tutor portal, video course hosting, multi-tenant billing, public candidate registration
Change management	New requirements must be recorded, reviewed for impact, and incorporated into a revised PRD version

Table of contents

1. Executive summary
2. Product vision, objectives and success measures
3. Scope and exclusions
4. Users, roles and permissions
5. Information architecture and navigation
6. Functional requirements — Admin Portal
7. Functional requirements — Candidate Portal
8. Shared platform capabilities
9. Core workflows
10. Business rules
11. Data model and integrity requirements
12. Technical architecture guidance
13. Folder structure requirements
14. UI/UX and responsive requirements
15. Security, privacy and reliability
16. Non-functional requirements
17. Notifications and email behavior
18. Reporting and auditability
19. Implementation sequence and definition of done
20. Testing and acceptance strategy
21. MVP release criteria
22. Assumptions, open decisions and risks

Appendix A. Suggested API domains

Appendix B. Builder instructions / implementation guardrails

1. Executive summary

Kodewar digital learn is a company-operated digital learning and candidate management platform. It centralizes candidate enrollment, course and batch organization, tutor assignment, meeting scheduling, attendance, learning resources, and candidate communications.

The product has two portals: an Admin Portal operated by exactly one Super Admin, and a Candidate Portal used by enrolled candidates. Tutors are managed as records by the Super Admin and do not receive a separate portal in the MVP.

The platform is not a multi-tenant SaaS product in its first release. It serves one company and should be architected cleanly enough to allow future evolution without implementing tenant-management complexity now.

2. Product vision, objectives and success measures

2.1 Vision

Provide a dependable, responsive, and easy-to-operate digital workspace for coordinating company training programs and giving candidates a clear view of their schedules, resources, notifications, and attendance.

2.2 Business objectives

- Centralize candidate records and enrollment information.
- Reduce manual work involved in scheduling and rescheduling training sessions.
- Communicate meeting changes and learning updates reliably.
- Maintain accurate batch, tutor, session, and attendance records.
- Make learning documents and resources available to the correct audience.
- Give candidates a simple self-service portal without exposing administrative controls.
- Provide traceability for important administrative actions.
- Support mobile, tablet, and desktop usage without losing core functionality.

2.3 Suggested success measures

Measure	Suggested measurement
Candidate import quality	Successful import rate; rejected rows categorized with actionable validation messages
Scheduling reliability	Sessions created without invalid time ranges or conflicting tutor assignments
Notification reliability	Queued, sent, delivered, bounced, and failed email counts where provider data is available
Attendance completeness	Percentage of completed sessions with attendance submitted
Resource access	Candidates can access only resources assigned to their current course/batch or explicitly permitted audience
Application stability	No release-blocking errors or page crashes in critical workflows
Responsive usability	Critical workflows usable at supported mobile, tablet, and desktop widths

3. Scope and exclusions

3.1 In scope for MVP

- Exactly one Super Admin account.

- Candidate Portal with email-or-Candidate-ID login.
- Individual candidate creation and bulk CSV/XLSX import.
- Course, batch, tutor, and candidate assignment management.
- Candidate transfer between batches with assignment history.
- Individual and recurring meeting scheduling.
- Manual meeting access enable/disable by Super Admin.
- Custom reminder intervals and email/in-app notifications.
- Manual attendance recording by Super Admin.
- Document and resource upload, assignment, preview where supported, and download.
- Candidate dashboard, meetings, resources, notifications, attendance summary, and profile.
- Audit logs, email logs, reporting, and essential system settings.

3.2 Explicitly out of scope for MVP

- Separate tutor portal or tutor self-service login.
- Video hosting, video lessons, streaming, and video progress tracking.
- Multi-company registration, tenant switching, and tenant billing.
- Public candidate self-registration.
- Automatic attendance based on Google Meet presence.
- Automatic Google Meet creation unless separately approved and integrated.
- Candidate self-marking of attendance.
- Candidate ability to move their own course or batch.
- Discussion forums, chat, assignment grading, quizzes, and certificates unless added through a later approved scope change.

4. Users, roles and permissions

4.1 Super Admin

There is exactly one Super Admin account in the initial release. It has full administrative authority over candidates, courses, batches, tutors, meetings, attendance, resources, notifications, reports, and settings.

- Admin account is provisioned through a controlled setup or deployment process; public admin registration is disabled.
- Admin can create, edit, deactivate, and archive business records where appropriate.
- Admin can import candidates and review import errors before committing valid records.
- Admin can enable and disable meeting access.
- Admin can mark and correct attendance, with correction reasons and audit history.
- Admin can view platform-level operational reports and audit logs.

4.2 Candidate

A candidate can access only their own profile, active enrollment information, authorized meetings, notifications, attendance history, and resources assigned to their current eligible course/batch or explicitly allowed audience.

- Candidate can log in using registered email address OR unique Candidate ID and password.
- Candidate must replace temporary credentials at first login before normal portal access.
- Candidate cannot alter administrative fields such as Candidate ID, course, batch, enrollment status, attendance, or meeting schedule.

- Candidate can edit only profile fields explicitly enabled by the company.

4.3 Tutor records

Tutors are managed by the Super Admin as operational records. They have no portal account or direct access to the application in MVP. Tutor data may include name, email, phone, specialization, status, and assigned courses.

5. Information architecture and navigation

5.1 Admin Portal navigation

Navigation item	Primary purpose
Dashboard	Operational overview, counts, upcoming sessions, recent activity
Candidates	Individual creation, bulk import, search, filters, status, enrollment history
Courses	Create and maintain course catalog
Batches	Create batches, set capacity/dates, assign course, view candidates
Tutors	Maintain tutor records and course associations
Meeting Scheduler	Calendar/list, create, edit, reschedule, cancel, recurring series and occurrence controls
Attendance	Select session, mark attendance, amend with reason, export records
Resources	Upload, categorize, assign, publish, archive, and manage files/links
Notifications	Announcements and notification management
Reports	Candidate, batch, course, attendance, meeting, tutor and resource reports
Audit Logs	Review administrative activity
Settings	Company profile, timezone, reminder defaults, file limits, email configuration and security settings

5.2 Candidate Portal navigation

Navigation item	Primary purpose
Dashboard	Personalized overview, next meeting, announcements, recent resources, attendance summary
My Meetings	Upcoming, access-enabled, cancelled, and completed sessions
Documents & Resources	Assigned materials with search, categories, preview and download
My Attendance	Attendance percentage and session-wise history
Notifications	In-app notices, read/unread state, timestamps
My Profile	Personal details and enrollment information
Account Settings	Change password, password recovery, logout

6. Functional requirements — Admin Portal

6.1 Admin dashboard

The dashboard must load real values from persisted application data. It must not display hardcoded sample counts or fake recent activity.

- Show total candidates, active candidates, courses, active batches, and active tutors.
- Show today's meetings, upcoming meetings, cancelled sessions, and completed sessions.

- Show attendance summary for a configurable period.
- Show recent imports, recent administrative actions, and recent resource uploads.
- Provide shortcuts to common actions such as Add Candidate, Import Candidates, Schedule Meeting, and Upload Resource.
- Support loading, empty, partial-failure, and error states without crashing.
- Dashboard cards and charts must be based on authorized backend queries and update after successful mutations.

6.2 Candidate management

Individual candidate creation

Admin can create a candidate through a validated form. Suggested fields:

Field	Requirement
Candidate ID	Unique identifier; generated by system or entered by Admin subject to uniqueness validation
Full name	Required
Email	Required; normalized and unique
Phone	Optional or company-configurable; validate format
Course	Required valid active course
Batch	Required valid active batch belonging to selected course
Enrollment date	Default from server date, editable by Admin if authorized
Status	Pending Activation, Active, Inactive, or Archived
Remarks	Optional internal notes; not visible to candidate unless explicitly intended

- On successful creation, create candidate account and enrollment assignment atomically or with a recoverable transaction/outbox workflow.
- Generate a secure temporary credential or single-use setup flow according to configured activation method.
- Queue an account activation email and record the outcome.
- Do not report account creation as failed solely because email delivery is temporarily unavailable; show account state and email delivery state separately.

Bulk import

Support CSV and XLSX import with a downloadable template, field mapping if needed, validation preview, row-level errors, duplicate detection, and explicit confirmation.

1. Admin downloads the current import template.
2. Admin uploads a supported CSV/XLSX file.
3. System checks file type, size, columns, required fields, data types, and row count limits.
4. System validates each row, including email uniqueness, Candidate ID uniqueness, course and batch references, and one-active-batch rule.
5. System presents valid rows, invalid rows, duplicates, and warnings separately.
6. Admin can download an error report and correct the source file.
7. Admin confirms import; valid rows are committed according to the selected import policy.
8. System generates account activation credentials and queues emails for newly created candidates.
9. System displays a final import summary and provides a durable import history.

Import must be idempotent where possible. Re-uploading the same file must not silently create duplicate candidates. The product owner must choose whether duplicate rows are rejected, skipped, or updated; default MVP behavior should be

reject/skip with clear row-level reporting, not silent overwrite.

Candidate search and lifecycle

- Search by name, Candidate ID, email, phone, course, or batch where permitted.
- Filter by course, batch, activation status, enrollment date, and candidate status.
- View candidate profile, current batch, assignment history, meetings, attendance, and resource eligibility.
- Edit candidate details with validation and audit logging.
- Deactivate or archive candidates without deleting historical records.
- Resend activation instructions or initiate credential reset.
- Export filtered candidate data with permission checks and audit logging.

6.3 Course management

- Create, edit, activate, deactivate, and archive courses.
- Course fields: course ID/code, name, description, duration, status, optional start/end dates, and metadata.
- Course names/codes must be unique according to the agreed uniqueness policy.
- Course cannot be hard-deleted if referenced by candidates, batches, sessions, or resources; archive/deactivate instead.
- Course selection should constrain batch choices in candidate forms and imports.

6.4 Batch management and candidate allocation

- Create and edit batches with batch name/code, course, start/end dates, capacity, status, and notes.
- Each batch belongs to one course in the MVP unless a later requirement explicitly permits cross-course batches.
- View candidate roster and batch schedule.
- Assign candidates individually or in bulk.
- Transfer candidates from one batch to another with effective date, reason, and confirmation.
- Enforce no more than one active batch assignment per candidate at any point in time.
- Preserve prior assignments with start date, end date, status, and transfer reason.
- Future meeting eligibility must use the candidate's active assignment at the relevant time; historical attendance must remain unchanged.
- Warn Admin if a transfer affects upcoming sessions, reminders, or resource access.

6.5 Tutor management

- Create, edit, deactivate, and search tutor records.
- Fields: tutor ID, full name, email, phone, specialization, status, assigned courses, and internal notes.
- Assign tutors to courses and meetings.
- Show tutor's upcoming and historical sessions.
- Validate overlapping sessions for the same tutor unless Admin explicitly overrides with a recorded reason.
- Tutor records do not create login accounts in MVP.

6.6 Meeting scheduler and management

Meeting fields

Field	Requirement
Title	Required session title
Course	Valid active course
Batch	Valid batch under selected course
Tutor	Assigned active tutor
Description	Optional session purpose or instructions
Date	Required local calendar date
Start / end time	Required; end must be after start
Timezone	Use configured company timezone and persist timestamps consistently
Meeting platform	Google Meet or another approved external platform
Meeting URL	Validated URL; access is protected by backend authorization
Session type	Individual occurrence or generated recurring occurrence
Status	Scheduled, Cancelled, Completed; any additional state must be explicitly modeled
Access status	Disabled or Enabled, independently controlled by Admin
Reminder configuration	One or more configured offsets and delivery channels

Individual scheduling

1. Admin opens Schedule Meeting.
2. Admin selects course and batch.
3. System loads valid candidates through the batch roster; Admin does not manually type a candidate list for normal batch sessions.
4. Admin selects date, start/end time, tutor, platform, and URL.
5. System validates time range, tutor conflicts, batch conflicts, and required fields.
6. Admin saves the meeting.
7. System persists the session and queues notifications for candidates who are eligible in the selected batch.
8. Candidate dashboard and My Meetings show the session with access disabled until Admin enables it.

Recurring scheduling

Support daily, weekly, selected weekdays, monthly, and custom interval recurrence where feasible. Admin must set a start date and either an end date or a maximum number of occurrences. The UI must preview generated occurrences before confirmation.

- Store recurrence definition separately from generated meeting sessions.
- Generate individual session records with their own IDs, dates, URLs, access status, status, reminders, and attendance.
- Support editing one occurrence, this and future occurrences, or the entire series.
- Support cancelling one occurrence or the series/future occurrences.
- Do not mutate completed historical sessions when editing a series.
- Allow a per-occurrence tutor or URL override if required.
- Handle daylight-saving changes and company timezone rules if the chosen timezone observes them.
- Prevent duplicate generation through idempotency keys or unique occurrence identifiers.

Rescheduling and cancellation

- Admin can update date, time, tutor, URL, and description.
- Require a reschedule/cancellation reason where configured.
- Notify only affected candidates and record notification delivery state.
- Recalculate pending reminders after a reschedule.
- Cancel pending reminders after cancellation.
- Preserve previous schedule details in meeting history or audit records.
- Candidate views must show the latest authoritative schedule.

Manual meeting access control

Meeting access is not automatically enabled by scheduled start time. Admin explicitly enables or disables access for each occurrence.

- Default access status is Disabled.
- Enable action makes the Join Meeting control available to eligible candidates.
- Disable action immediately revokes application-mediated access to the meeting URL.
- Access state changes are persisted server-side and audited with actor and timestamp.
- Cancelled sessions cannot be enabled.
- Completed sessions remain non-joinable unless Admin performs a separately authorized reopen action, if that capability is approved.
- Do not expose raw meeting URLs in client payloads while access is disabled. The backend should return the URL only when the candidate is authorized and access is enabled.
- Explain in the UI that once a candidate has copied or opened an external meeting URL, the application cannot revoke the external platform's independently issued access.

6.7 Attendance management

Attendance is manually marked by the Super Admin for a specific meeting occurrence.

- Admin selects course, batch, and meeting occurrence.
- System loads candidates eligible for that session based on enrollment/assignment rules.
- Admin marks each candidate Present, Absent, Late, or Excused.
- Admin can use bulk actions such as Mark all present, followed by individual corrections.
- Admin can add remarks and save attendance.
- Admin can edit a saved record; require a reason for corrections and write an audit event.
- Prevent duplicate attendance rows for the same candidate and meeting.
- Candidate can view their own session-wise attendance and calculated summary.
- Define whether Late and Excused count toward attendance percentage in configurable policy; do not silently assume.

Suggested formula: Attendance percentage = qualifying attended sessions ÷ eligible sessions × 100. The company must define whether Late counts as attended and whether Excused sessions are excluded from the denominator.

6.8 Resource and document management

- Upload one or multiple files and create external resource links.
- Supported types should be configurable; suggested initial types: PDF, DOC/DOCX, XLS/XLSX, PPT/PPTX, common image formats, and ZIP.
- Fields: title, description, category, file or URL, course assignment, batch assignment, publication status, uploaded by, and timestamps.

- Allow assignment to a course, a batch, or an explicitly defined audience. If both course and batch are selected, define whether access is intersection or union; recommended default is the narrower intersection.
- Support draft/unpublished, published, archived, and deleted/soft-deleted states.
- Candidate can preview supported formats and download authorized files.
- Use private object storage and short-lived signed download URLs or equivalent authorization.
- Validate file extension and MIME type; enforce file-size limits; scan uploads for malware where supported.
- Never trust client-supplied file names or MIME types.
- Do not expose storage bucket paths or public permanent URLs for private resources.

6.9 Notifications and announcements

- Create announcements for all candidates or a selected batch/course audience.
- Support in-app notifications and email delivery where enabled.
- Candidate can view, mark read, and mark all read.
- Store created time, recipient, type, message, read status, related entity, and delivery state where applicable.
- Prevent notification leakage between candidates.
- Support pagination and unread count.
- Do not treat a queued email as delivered; show accurate status.

7. Functional requirements — Candidate Portal

7.1 Dashboard

- Show candidate name and relevant welcome message.
- Show current course and active batch.
- Show next upcoming meeting and meeting status.
- Show Join Meeting only when access is enabled by Admin.
- Show recent notifications and unread count.
- Show recently published resources assigned to the candidate.
- Show attendance summary.
- Use empty states when the candidate has no upcoming meetings or resources.
- Refresh or revalidate important meeting access state so Admin changes are reflected promptly.

7.2 My Meetings

- Show upcoming, access-enabled, cancelled, and completed meetings in clearly separated views or filters.
- Display title, date, start/end time, tutor, course/batch, description, and access state.
- Join button is disabled while access is Disabled, session is Cancelled, or session is Completed.
- Join button becomes available only when backend confirms the candidate is eligible and access is Enabled.
- Do not rely only on disabled UI controls; enforce access in the backend.
- Show reschedule updates and relevant cancellation messages.
- Use the company timezone consistently and label it clearly.

7.3 Documents and resources

- List only resources the candidate is authorized to access.
- Search by title and filter by category/type/course where appropriate.

- Show description, upload/publication date, file type, and size where available.
- Preview supported formats safely; otherwise provide download.
- Handle expired links, removed resources, and permission errors gracefully.

7.4 Notifications

- List in-app notifications in reverse chronological order.
- Show unread/read state and timestamps.
- Allow mark-as-read and mark-all-as-read.
- Open related meeting/resource details when the notification includes a valid authorized reference.
- Do not allow a candidate to read another candidate's notifications.

7.5 My Attendance

- Show attendance percentage using the approved company calculation policy.
- Show session-wise date, title, and attendance status.
- Clearly distinguish absent, late, excused, and present.
- Do not allow candidates to modify attendance.

7.6 Profile and account settings

- Show Candidate ID, name, registered email, phone (if held), course, active batch, enrollment date, and status.
- Allow editing only approved personal fields.
- Course, batch, Candidate ID, activation status, and attendance remain admin-controlled.
- Allow password change with current-password verification.
- Provide forgot-password flow through registered email.
- Provide secure logout and session invalidation.

8. Shared platform capabilities

8.1 Authentication and authorization

- Use secure password hashing with a modern adaptive algorithm such as Argon2id or bcrypt with suitable cost parameters.
- Use secure, HttpOnly, SameSite cookies or an appropriately secured token strategy; avoid storing long-lived access tokens in localStorage.
- Apply server-side authorization to every protected API and file operation.
- Use rate limiting and account protection for login and password reset endpoints.
- Use generic login failure messages to reduce account enumeration.
- Enforce candidate ownership checks on every candidate-scoped resource.
- Enforce exactly one Super Admin account through provisioning logic and database constraints where practical.

8.2 Email service

Use a transactional email provider through a backend service abstraction. Provider credentials must be stored in environment secrets, never in source code or frontend bundles.

- Templates for activation, password reset, meeting scheduled, rescheduled, cancelled, reminder, batch transfer, resource published, and announcement.
- Queue email delivery asynchronously.
- Track queued, sent, delivered/bounced/failed states when provider webhooks support them.

- Retry transient failures with bounded backoff and dead-letter handling.
- Use an outbox pattern or equivalent to avoid losing notification events when a database transaction succeeds but a process fails.
- Never log plaintext passwords, reset tokens, or full private file URLs.

8.3 File storage

Store uploaded files in private object storage or a secure equivalent. Store metadata and storage keys in the database; do not store large binary files directly in relational rows unless a deliberate technical decision is made.

8.4 Time and timezone

- Configure one company timezone in system settings.
- Persist timestamps in UTC and convert for display using the company timezone.
- Validate local date/time input and daylight-saving ambiguity if applicable.
- Reminder scheduling and recurring occurrence generation must use the configured timezone.
- Use server time for access-control decisions; never trust a candidate device clock.

9. Core workflows

9.1 Candidate import and activation

1. Admin downloads the template and populates candidate rows.
2. Admin uploads the file and reviews validation results.
3. Admin confirms import.
4. System creates candidate, enrollment, and batch assignment records.
5. System creates activation credentials or a one-time setup invitation.
6. Email job is queued and logged.
7. Candidate receives instructions, signs in, and replaces the temporary credential.
8. Candidate can then access the Candidate Portal according to their active status.

9.2 Meeting lifecycle

1. Admin creates an individual meeting or recurring series.
2. System validates batch, tutor, date/time, and recurrence.
3. System creates meeting session records and reminder jobs.
4. System sends schedule notification to eligible candidates.
5. Meeting appears in Candidate Portal with access disabled.
6. Admin manually enables access when ready.
7. Candidate joins through an authorized backend-mediated link response.
8. Admin disables access and/or marks the session completed.
9. Admin records attendance; candidates see their own results.

9.3 Batch transfer

1. Admin opens candidate or batch roster and chooses Transfer.
2. Admin selects destination batch and effective date.
3. System validates course compatibility, batch status/capacity, and one-active-batch constraint.
4. Admin confirms transfer and enters a reason if required.

5. System closes the previous assignment and creates a new assignment.
6. System recalculates future meeting/resource eligibility.
7. System notifies the candidate and records an audit event.
8. Historical sessions and attendance remain unchanged.

10. Business rules

ID	Rule
BR-01	The product serves one company in MVP; no tenant registration or tenant switching.
BR-02	Exactly one Super Admin account is allowed in the initial release.
BR-03	Public Super Admin registration is disabled.
BR-04	Candidate login accepts registered email or unique Candidate ID plus password.
BR-05	Candidate email and Candidate ID must be unique.
BR-06	Candidate accounts are created by Admin, not public self-registration.
BR-07	New candidates receive temporary credentials or a one-time activation flow and must set a permanent password before normal access.
BR-08	A candidate may have only one active batch assignment at a time.
BR-09	Batch transfers preserve historical assignments and attendance.
BR-10	A batch belongs to one course in MVP unless later changed by approved scope.
BR-11	A meeting must have a valid start and end time; end must be later than start.
BR-12	A meeting session belongs to a valid batch and has an assigned tutor where required.
BR-13	Individual and recurring meeting scheduling are supported.
BR-14	Recurring series and individual meeting occurrences are stored separately.
BR-15	Every recurring occurrence has independent access, status, reminders, and attendance.
BR-16	Meeting access defaults to Disabled.
BR-17	Meeting start time does not automatically enable meeting access.
BR-18	Only Super Admin can enable or disable meeting access.
BR-19	Cancelled and completed meetings are not joinable.
BR-20	Meeting URL is returned to candidates only when the candidate is eligible and access is enabled.
BR-21	Meeting access changes are audited.
BR-22	Rescheduling updates candidate views and pending reminders.
BR-23	Cancelled meetings do not trigger future reminders.
BR-24	Reminder intervals and channels are configurable.
BR-25	Attendance is manually recorded by Super Admin.
BR-26	Attendance is unique per candidate per meeting occurrence.
BR-27	Attendance corrections require audit history and, preferably, a reason.
BR-28	Candidates can view but cannot modify attendance.
BR-29	Candidates only access resources assigned to their permitted audience.
BR-30	Tutors do not have a portal or login account in MVP.
BR-31	Important records are archived rather than hard-deleted when referenced by history.
BR-32	Email delivery state is tracked separately from business-record creation.
BR-33	All timestamps are handled consistently using the configured company timezone.
BR-34	All authorization checks are enforced server-side.

11. Data model and integrity requirements

11.1 Proposed core entities

Entity	Purpose / key relationships
Admin	The single Super Admin identity and authentication data.
Candidate	Candidate profile, Candidate ID, email, password hash, status.
Course	Course catalog and status.
Batch	Belongs to a course; contains candidates through assignment history.
Tutor	Tutor profile and operational status; no login.
CandidateBatchAssignment	Candidate-to-batch history with effective dates and one active assignment constraint.
MeetingSeries	Recurrence rule, timezone, series-level defaults, start/end limits.
MeetingSession	A specific occurrence with date/time, tutor, URL, status, and access state.
MeetingReminder	Reminder offset, channel, scheduled time, delivery state, idempotency key.
Attendance	Candidate + meeting session + status + remarks + correction metadata.
Resource	File/link metadata, status, owner, category.
ResourceAssignment	Resource audience mapping to course, batch, or explicitly defined audience.
Notification	Recipient, type, content, read state, related entity.
EmailLog	Recipient, template, provider ID, delivery state, timestamps; no secrets.
AuditLog	Actor, action, entity, entity ID, timestamp, request/correlation ID, safe change summary.

11.2 Data integrity and deletion

- Use foreign keys and unique constraints for identifiers and relationships.
- Use transactions for multi-record operations such as import commits, candidate transfer, and meeting-series generation.
- Use soft deletion/archive for referenced business records.
- Do not cascade-delete attendance, meeting history, or audit logs when a candidate/course/batch is archived.
- Use database constraints to prevent duplicate attendance and multiple active batch assignments.
- Keep credentials, tokens, and sensitive secrets out of logs.
- Use migrations with version control; do not manually alter production schema without a migration.

12. Technical architecture guidance

The technology stack is not yet formally selected. The following is a recommended implementation direction, subject to confirmation by the engineering team.

Layer	Suggested direction	Requirements
Frontend	React or Next.js + TypeScript	Responsive portals, shared component library, typed API client, route guards, accessible forms.
Backend	NestJS or another structured Node.js backend	Modular domain services, validation, authorization, transaction boundaries, API documentation.
Database	PostgreSQL	Relational integrity, migrations, indexes, constraints, backups.
File storage	Private S3-compatible object storage	Private objects, signed URLs, size/type validation, malware scanning where available.

Layer	Suggested direction	Requirements
Email	Transactional email provider	Templates, queue, delivery webhooks, retry, suppression handling.
Background jobs	Redis-backed queue or managed job service	Reminders, email, recurring session generation, retries, idempotency.
Monitoring	Structured logs and error monitoring	Frontend/backend error reporting, health checks, request correlation.
Deployment	Managed cloud hosting	Separate environments, secrets management, automated deployment and rollback.

Use a modular monolith for the MVP unless there is a demonstrated need for microservices. This reduces operational complexity while keeping domain boundaries clear.

13. Folder structure requirements

The repository must be organized neatly around the Admin, Candidate, and Common areas. Shared logic must not be duplicated between portals. Keep domain modules, reusable UI, API services, types, validation schemas, and tests clearly separated.

```
kodewar-digital-learn/
■■■ apps/
■ ■■■ web/
■ ■ ■■■ src/
■ ■ ■ ■■■ admin/
■ ■ ■ ■ ■■■ pages-or-routes/
■ ■ ■ ■ ■■■ components/
■ ■ ■ ■ ■■■ features/
■ ■ ■ ■ ■ ■■■ dashboard/
■ ■ ■ ■ ■ ■■■ candidates/
■ ■ ■ ■ ■ ■■■ courses/
■ ■ ■ ■ ■ ■■■ batches/
■ ■ ■ ■ ■ ■■■ tutors/
■ ■ ■ ■ ■ ■■■ meetings/
■ ■ ■ ■ ■ ■■■ attendance/
■ ■ ■ ■ ■ ■■■ resources/
■ ■ ■ ■ ■ ■■■ notifications/
■ ■ ■ ■ ■ ■■■ reports/
■ ■ ■ ■ ■■■ layouts/
■ ■ ■ ■■■ candidate/
■ ■ ■ ■ ■■■ pages-or-routes/
■ ■ ■ ■ ■■■ components/
■ ■ ■ ■ ■■■ features/
■ ■ ■ ■ ■ ■■■ dashboard/
■ ■ ■ ■ ■ ■■■ meetings/
■ ■ ■ ■ ■ ■■■ resources/
■ ■ ■ ■ ■ ■■■ attendance/
■ ■ ■ ■ ■ ■■■ notifications/
■ ■ ■ ■ ■ ■■■ profile/
■ ■ ■ ■ ■■■ layouts/
■ ■ ■ ■■■ common/
■ ■ ■ ■■■ components/
■ ■ ■ ■■■ hooks/
■ ■ ■ ■■■ services/
■ ■ ■ ■■■ api/
```

■ ■ ■ ■■■ auth/
■ ■ ■ ■■■ utils/
■ ■ ■ ■■■ constants/
■ ■ ■ ■■■ types/
■ ■ ■ ■■■ validation/
■ ■ ■ ■■■ styles/
■ ■ ■ ■■■ accessibility/
■ ■ ■■■ public/
■ ■■■ api/
■ ■■■ src/
■ ■■■ modules/
■ ■ ■■■ auth/
■ ■ ■■■ admins/
■ ■ ■■■ candidates/
■ ■ ■■■ courses/
■ ■ ■■■ batches/
■ ■ ■■■ tutors/
■ ■ ■■■ enrollments/
■ ■ ■■■ meetings/
■ ■ ■■■ attendance/
■ ■ ■■■ resources/
■ ■ ■■■ notifications/
■ ■ ■■■ email/
■ ■ ■■■ reports/
■ ■ ■■■ audit/
■ ■ ■■■ settings/
■ ■■■ common/
■ ■ ■■■ guards/
■ ■ ■■■ decorators/
■ ■ ■■■ filters/
■ ■ ■■■ interceptors/
■ ■ ■■■ middleware/
■ ■ ■■■ validators/
■ ■ ■■■ utils/
■ ■■■ config/
■ ■■■ main.ts
■■■ packages/
■ ■■■ shared-types/
■ ■■■ shared-validation/
■ ■■■ shared-config/
■■■ database/
■ ■■■ migrations/
■ ■■■ seeds/
■ ■■■ schema/
■■■ tests/
■ ■■■ e2e/
■ ■■■ integration/
■ ■■■ accessibility/
■ ■■■ fixtures/
■■■ docs/
■ ■■■ prd/
■ ■■■ architecture/
■ ■■■ api/
■ ■■■ runbooks/

```
■■■ .env.example
■■■ README.md
■■■ package.json
```

Folder names may be adapted to the chosen framework, but the Admin, Candidate, and Common separation is mandatory. Do not put secrets, production data, or uploaded user files in the source repository.

14. UI/UX and responsive requirements

14.1 Responsive behavior

- All screens must work on mobile, tablet, laptop, and desktop.
- Use mobile-first layout rules and fluid containers.
- Admin tables must become usable card/list layouts or horizontally scroll within a clearly bounded region on small screens.
- Navigation must collapse into an accessible mobile menu or drawer.
- Forms must use touch-friendly controls, clear labels, and inline validation.
- Primary actions must remain reachable without horizontal page scrolling.
- Calendar and scheduler views must provide a mobile-friendly list/agenda alternative.
- Modals and drawers must fit within viewport height and support scrolling.
- File upload, preview, attendance marking, and meeting access must be usable on touch devices.

14.2 Design consistency

- Use a shared design system for typography, spacing, colors, buttons, inputs, dialogs, tables, cards, badges, and empty states.
- Admin and Candidate portals may have distinct navigation but should share common tokens and components.
- Use consistent loading skeletons, error messages, confirmation dialogs, and success feedback.
- Provide accessible focus states and keyboard navigation.
- Meet WCAG 2.1 AA as a target where practical.

14.3 Reliability of pages

- Every route must have a loading state, empty state, error state, and unauthorized/not-found state as applicable.
- One failing widget must not crash the entire page; isolate recoverable feature errors.
- Validate API responses and handle null/empty data safely.
- Do not assume arrays, records, or user profiles are always present.
- Prevent duplicate form submission and show pending state.
- Use error boundaries at application and feature boundaries.
- Preserve user input when a recoverable server error occurs, where safe.

15. Security, privacy and reliability

- Enforce authentication and authorization on the server, not only in the frontend.
- Protect against SQL injection, XSS, CSRF, SSRF, insecure direct object references, and unsafe file uploads.
- Validate and sanitize all user input on the backend.
- Use HTTPS in every deployed environment.
- Store secrets in environment/secret management; never hardcode credentials or API keys.
- Apply least privilege to database, storage, email, and deployment credentials.

- Rate-limit authentication, imports, file uploads, and sensitive endpoints.
- Use secure password reset and activation tokens with expiration and single-use semantics.
- Log administrative events while excluding passwords, tokens, and sensitive personal data not required for audit.
- Back up database and file metadata; document restore procedures and test recovery.
- Provide a privacy notice and data retention policy appropriate to company requirements and applicable law.

16. Non-functional requirements

Category	Requirement
Performance	Common pages should load promptly under expected company usage; use pagination, indexes, caching where safe, and avoid unbounded list queries.
Scalability	Support growth in candidates, batches, sessions, files, and notification volume without redesigning every module.
Availability	Deploy with health checks, monitoring, backups, and documented recovery.
Reliability	Use transactions for critical multi-step writes and idempotent background jobs.
Security	Role/ownership checks, secure authentication, private file access, secrets management, and auditability.
Accessibility	Keyboard access, labels, focus visibility, contrast, semantic structure, and screen-reader support.
Maintainability	Modular architecture, typed contracts, code review, linting, formatting, tests, and migrations.
Compatibility	Support current stable versions of major browsers and responsive mobile browsers.
Observability	Structured logs, error tracking, request IDs, queue metrics, and email delivery state.
Data integrity	Database constraints, referential integrity, unique indexes, and safe archival.

17. Notifications and email behavior

Trigger	Audience	Channels
Candidate account created	New candidate	Email activation instructions
Meeting scheduled	Eligible batch candidates	Email + in-app
Meeting rescheduled	Affected candidates	Email + in-app
Meeting cancelled	Affected candidates	Email + in-app
Meeting reminder	Eligible candidates	Configured email and/or in-app
Batch transfer	Transferred candidate	Email + in-app
Resource published	Assigned audience	In-app; email if configured
Announcement published	Selected audience	In-app + optional email
Password reset	Requesting candidate/admin	Email

Notification creation and email delivery must be decoupled from the user-facing request where appropriate. A successful meeting save should not be rolled back merely because an external email provider is unavailable; the event should be queued and the failure made visible to Admin.

18. Reporting and auditability

18.1 Reports

- Candidate report: candidate profile, course, current batch, status, enrollment date.
- Batch report: batch capacity, candidate count, roster, transfer history.
- Course report: courses, batches, enrollments, active/inactive state.
- Attendance report: candidate/session statuses, percentage, date range, batch/course filters.
- Meeting report: upcoming, completed, cancelled, rescheduled sessions, tutor and batch filters.
- Tutor report: assigned courses, sessions, schedule conflicts.
- Resource report: uploads, publication status, audience, upload date.

Reports should support date filters and CSV/XLSX export. Exported data must respect authorization and avoid exposing unnecessary candidate personal information.

18.2 Audit logs

Record important administrative actions with actor, action, entity type, entity ID, timestamp, request/correlation ID, and safe before/after summary where appropriate.

- Candidate create/update/deactivate/archive/import.
- Course and batch create/update/archive.
- Candidate batch transfer.
- Tutor create/update/deactivate and assignment.
- Meeting create/update/reschedule/cancel and recurring-series changes.
- Meeting access enable/disable.
- Attendance create/update/correction.
- Resource upload/update/publish/archive/delete.
- Notification/announcement creation.
- Security-sensitive settings changes.

19. Implementation sequence and definition of done

The builder must implement one feature/module at a time. Do not start the next module merely because the current screen renders. A module is complete only after its frontend, backend, database persistence, validation, authorization, error handling, and tests work end-to-end.

Phase	Module	Exit gate before proceeding
0	Project foundation	Build runs; environments and secrets configured; lint/typecheck/tests pass; route and error boundaries exist.
1	Authentication	Admin and candidate login, email/Candidate ID lookup, session handling, password reset, first-login password change, authorization tests pass.
2	Courses and batches	CRUD, validation, archive, relationships, constraints, responsive UI and tests pass.
3	Candidate management/import	Individual create, CSV/XLSX import preview, validation, duplicate handling, activation email, batch assignment, history and tests pass.
4	Tutor management	Tutor CRUD, course association, meeting schedule visibility, conflict checks pass.
5	Meeting scheduling	Individual and recurring sessions, edit scopes, cancellation, rescheduling, timezone handling, conflict checks and tests pass.

Phase	Module	Exit gate before proceeding
6	Meeting access and reminders	Manual access control, backend URL protection, per-occurrence access, reminder rescheduling/cancellation and tests pass.
7	Candidate portal	Dashboard, meetings, protected join action, profile, responsive layouts and authorization tests pass.
8	Resources	Upload validation, private storage, audience assignment, preview/download, authorization and tests pass.
9	Attendance	Session roster, manual marking, correction reason, candidate history, calculation policy and tests pass.
10	Notifications and email	In-app and email triggers, queues, retries, logs, read state and tests pass.
11	Reports and audit	Filters, exports, audit coverage, permission checks and tests pass.
12	Release hardening	End-to-end regression, responsive, accessibility, security, backup/restore and deployment checks pass.

19.1 Module completion gate

- All required UI actions work; no dead buttons or placeholder interactions.
- Data is persisted in the real database and survives refresh/re-login.
- API validation and authorization are implemented server-side.
- Loading, empty, success, validation, network, and server error states are handled.
- No console errors or uncaught exceptions in the tested flow.
- No page crash on missing, empty, malformed, or unauthorized data.
- Unit/integration tests for core logic pass.
- End-to-end test for the main user journey passes.
- Responsive checks pass at mobile, tablet, and desktop breakpoints.
- Existing modules pass regression tests.

20. Testing and acceptance strategy

20.1 Required test layers

- Unit tests for validation, recurrence generation, attendance calculations, access rules, and business services.
- Integration tests for database constraints, transactions, API authorization, email queueing, and storage access.
- End-to-end tests for Admin and Candidate journeys.
- Responsive tests at representative mobile, tablet, and desktop viewports.
- Accessibility checks for keyboard navigation, focus order, labels, contrast, and dialogs.
- Security tests for IDOR, privilege escalation, session handling, file access, input validation, and rate limiting.
- Regression tests after every module integration.

20.2 Critical end-to-end acceptance scenarios

1. Super Admin can sign in; a candidate cannot access Admin routes or APIs.
2. Candidate can sign in using email and also using Candidate ID.
3. New candidate receives activation instructions and must replace temporary credentials.
4. Duplicate candidate email or Candidate ID is rejected with a useful message.
5. Bulk import preview identifies valid, invalid, and duplicate rows before commit.

6. Candidate cannot be assigned to two active batches.
7. Batch transfer closes prior assignment, creates new assignment, preserves attendance, and updates future eligibility.
8. Admin schedules an individual session and the eligible candidates see it.
9. Admin creates a recurring series and generated occurrences have independent records.
10. Admin edits one recurring occurrence without changing unrelated occurrences.
11. Meeting link remains unavailable at and after start time until Admin enables access.
12. Candidate cannot retrieve meeting URL through direct API call while access is disabled.
13. Admin disables meeting access and candidate can no longer obtain the URL through the application.
14. Rescheduling updates candidate view and pending reminder schedule.
15. Cancellation prevents join access and suppresses pending reminders.
16. Admin marks attendance and candidate sees only their own attendance.
17. Admin corrects attendance with a reason and audit entry.
18. Candidate cannot access a resource assigned to another batch.
19. Resource upload rejects disallowed or oversized files.
20. Email provider failure is visible in delivery logs and does not create duplicate business records on retry.
21. All core screens remain usable on mobile and do not crash on empty datasets.

21. MVP release criteria

The MVP is ready for controlled production release only when all of the following are satisfied:

- All in-scope functional requirements are implemented or explicitly deferred with product-owner approval.
- Exactly one Super Admin provisioning and access policy is enforced.
- Candidate authentication and activation flows pass end-to-end tests.
- Candidate import and batch allocation preserve data integrity.
- Individual and recurring meeting workflows pass all acceptance scenarios.
- Manual meeting access is enforced by the backend.
- Attendance is accurate, auditable, and visible to the correct candidate.
- Resource access is private and audience-restricted.
- Email and in-app notifications have observable delivery states.
- No critical or high-severity security issues remain open.
- No release-blocking page crashes or uncaught runtime errors remain.
- Responsive checks pass for all critical screens.
- Database backups and restore procedure are verified.
- Production environment secrets are configured outside source code.
- Monitoring, error reporting, and deployment rollback are available.

22. Assumptions, open decisions and risks

22.1 Current assumptions

- The company uses one configured timezone.
- Each batch belongs to one course in MVP.
- Candidates are assigned to one active batch at a time.

- Meeting URLs are supplied manually by Admin; Google Meet API integration is not assumed.
- Attendance is marked manually by Admin.
- Resources are documents/files and external links, not hosted video courses.
- One Super Admin is the only administrative role in MVP.
- Email provider, hosting, database, and final framework remain to be selected.

22.2 Open decisions to confirm before final schema freeze

Decision	Recommended default for planning
Can one course have multiple batches?	Yes; one course can have many batches.
Can a batch contain candidates from multiple courses?	No; one batch belongs to one course.
Candidate completion/exit lifecycle	Use Completed, Withdrawn, or Archived status; preserve history.
Recurring session Google Meet URL	Allow same URL by default but permit per-occurrence override.
Attendance correction	Allow Admin edit with mandatory reason and audit log.
Resource audience when both course and batch selected	Use intersection (narrowest audience) unless Admin explicitly chooses another audience mode.
Late/Excused attendance calculation	Make policy configurable; require product-owner confirmation.
Candidate communication replies	One-way announcements in MVP.

22.3 Key risks and mitigations

Risk	Mitigation
Temporary password exposure	Prefer single-use setup link; otherwise short expiry, forced replacement, secure generation and no plaintext storage.
Meeting URL leakage	Return URL only after backend access and ownership checks; explain external platform limitations.
Recurring schedule duplication	Use deterministic occurrence identity and idempotent generation.
Timezone mistakes	Persist UTC, retain configured timezone, test daylight-saving and boundary cases.
Incorrect candidate batch transfer	Use effective-dated assignment history and database constraints.
Email provider outage	Queue with retries, outbox pattern, and visible failure status.
Unauthorized file access	Private storage, signed URLs, ownership/audience checks, and file scanning.
Single Super Admin lockout	Provide a secure, documented account recovery/provisioning runbook without adding a second normal admin account.

Appendix A. Suggested API domains

Exact endpoint names and payload schemas should be finalized in API documentation. Suggested domain boundaries:

- Authentication: login, logout, refresh/session, forgot password, reset password, first-login password change.
- Admin: current admin profile, platform settings, account recovery administration.
- Candidates: create, update, get, search, import preview, import commit, export, activate/deactivate, resend activation.
- Courses: create, update, list, get, archive.
- Batches: create, update, list, roster, transfer candidate, assignment history.
- Tutors: create, update, list, deactivate, course assignment, schedule.

- Meetings: create individual, create series, list, get, update occurrence, update series, cancel, reschedule, enable access, disable access, complete.
- Attendance: session roster, bulk upsert, update record, candidate history, reports.
- Resources: upload initiation, upload completion, metadata CRUD, assignment, publish, archive, authorized preview/download.
- Notifications: list, unread count, mark read, mark all read, announcements.
- Reports: candidates, batches, courses, meetings, attendance, tutors, resources, exports.
- Audit: filtered audit log retrieval.

Appendix B. Builder instructions / implementation guardrails

The following instructions are mandatory for the AI application builder or development team:

- **Do not hardcode application data.** Do not place static candidate names, batches, courses, tutors, meetings, attendance figures, dashboard counts, notifications, or resource records in the application as fake operational data.
- **Use database-backed data.** All user-visible business data must be fetched from and persisted to the configured backend/database. Static UI labels, design tokens, fixed enum definitions, and documented defaults are acceptable; fake business records are not.
- **No silent mock mode.** Do not ship mocked API responses or local arrays as if they were real production records. If a development seed is needed, keep it in an explicit development-only seed process and never run it automatically in production.
- **Complete one functionality end-to-end.** Finish UI, API, database, validation, authorization, error handling, and tests before moving to the next feature.
- **No dead controls.** Every button, menu item, form, filter, tab, upload control, and action must perform its stated function or be clearly disabled with an explanation.
- **No page crashes.** Use route/feature error boundaries, defensive rendering, typed API responses, and explicit loading/empty/error states. Test null, empty, delayed, malformed, and unauthorized responses.
- **Neat folder organization.** Keep separate Admin, Candidate, and Common frontend areas, plus modular backend domains. Reuse shared components and avoid duplicated business logic.
- **Responsive by default.** Implement mobile-first layouts and verify all core workflows on phone, tablet, and desktop. Do not merely scale down desktop tables.
- **Server-side security.** Never rely on hidden buttons or frontend route guards as the only protection. Enforce ownership, role, meeting access, and resource permissions in the backend.
- **Do not hardcode secrets.** Use environment variables/secret management for database URLs, email provider credentials, storage credentials, signing keys, and other secrets. Include only placeholder variable names in .env.example.
- **Use migrations.** Database schema changes must be versioned and reproducible. Do not depend on manual production database edits.
- **Preserve history.** Avoid destructive deletion of candidates, batches, meetings, attendance, and audit records that are referenced by historical activity.
- **Test every integration.** Verify email, file storage, recurring jobs, reminders, and external meeting-link behavior with controlled test environments.
- **Prevent regressions.** Run automated tests for completed modules whenever new functionality is added. Fix regressions before proceeding.
- **Report honestly.** Do not mark a feature complete because its screen exists. Completion requires verified end-to-end behavior and passing acceptance checks.

End of PRD — Kodewar digital learn, version 1.2. This is a detailed implementation brief based on the product decisions confirmed to date. Open decisions in Section 22.2 should be resolved before final database schema freeze and production implementation.